

Reinforcement Learning Notes

Synthesized from the CS175 Resource Links

This document extracts the actual reinforcement learning content that is taught across the linked tutorials, libraries, courses, and books from the CS175 Canvas page. It is not a catalog of resource types. Instead, it turns the linked material into a set of study notes organized around the ideas of reinforcement learning itself.

The sources behind this synthesis include Sutton and Barto, David Silver's UCL course, Denny Britz's notebook-based repo, OpenAI Spinning Up, Berkeley's deep RL course, the Francois-Lavet deep RL monograph, Csaba Szepesvari's theory text, Bertsekas's dynamic-programming and control materials, and the library documentation for RLLib, MushroomRL, Baselines, and Coach.

1. The Agent, the Environment, and the Learning Objective

Reinforcement learning begins with repeated interaction. At time step t , the agent receives a state or observation, chooses an action, and the environment responds with both a scalar reward and a next state. The key point is that the agent is not given labeled correct actions. Instead, it must learn from consequences. That is what makes RL different from ordinary supervised learning: the learner's actions change its future data, so learning and control are inseparable.

The quantity the agent tries to maximize is not immediate reward alone, but **return**, the total future reward from a time step onward. In the discounted case,

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots,$$

where $\gamma \in [0, 1]$ is the discount factor. Discounting makes delayed rewards count less, keeps continuing problems mathematically well-defined, and encodes the preference for sooner reward over later reward. Once return is the objective, a locally bad action can still be globally good if it improves the future enough.

This agent-environment-return framing is the common starting point across Sutton and Barto, Silver's first two lectures, and Britz's introductory notes. The same loop also sits underneath all of the library APIs. Whether a framework is simple or distributed, it is still formalizing the same cycle of observing, acting, receiving reward, and updating.

Sources: <http://www.incompleteideas.net/book/RLbook2018.pdf>,
<https://www.davidsilver.uk/teaching/>, <https://github.com/dennybritz/reinforcement-learning>,
https://docs.ray.io/en/latest/rllib/package_ref/env.html,
https://mushroomrl.readthedocs.io/en/latest/source/mushroom_rl.agent_environment_interface.html

2. States, Markov Decision Processes, and Value Functions

The next conceptual step is deciding what information the agent should treat as its state. In the cleanest setting, the chosen state has the **Markov property**, meaning that once the current state is known, the future depends only on that state and the chosen action, not on the full past history. A control problem with states, actions, transition-reward dynamics, and discount is then modeled as a **Markov Decision Process**, or MDP.

Given an MDP and a policy π , the central objects are the **state value** $v_\pi(s)$ and the **action value** $q_\pi(s, a)$. The state value is the expected return starting from state s and following policy π . The action value is the expected return starting from s , first taking action a , and then following π . Optimal control asks for a policy whose values dominate all alternatives, producing the optimal value functions v_* and q_* .

These definitions matter because they turn the vague goal of "do well over time" into something the agent can estimate and optimize. Much of RL can be understood as different ways of approximating or improving these value functions under different assumptions about what is known, what is sampled, and what function class is being used.

Sources: <http://www.incompleteideas.net/book/RLbook2018.pdf>,
<https://www.davidsilver.uk/teaching/>, <https://raw.githubusercontent.com/dennybritz/reinforcement-learning/master/MDP/README.md>, <https://sites.ualberta.ca/~szepesva/papers/RLAlgsInMDPs.pdf>

3. Bellman Equations, Dynamic Programming, and Generalized Policy Iteration

The Bellman equations are the mathematical backbone of RL. They express a value recursively as immediate reward plus discounted continuation value. For a fixed policy, the Bellman expectation equation defines v_π by averaging over the policy's actions and the environment's stochastic transitions. For optimal control, the Bellman optimality equation replaces that policy average with a maximum over actions. This is the bridge from value estimation to actual decision making.

Dynamic programming assumes that the transition-reward model is known exactly. Under that assumption, policy evaluation repeatedly applies Bellman expectation backups to compute v_π , and policy improvement makes the policy greedy with respect to that value function. Alternating those steps gives **policy iteration**. Repeatedly applying the Bellman optimality operator gives **value iteration**. These methods show the ideal structure of RL clearly, but they become impractical when the model is unknown or the state space is too large.

Sutton and Barto's notion of **generalized policy iteration** is the broader synthesis. Most RL algorithms can be seen as incomplete, interleaved forms of evaluation and improvement. Bertsekas and Szepesvari push this idea further by viewing value iteration, policy iteration, actor-critic, LSPI-style methods, and even some rollout procedures as variants of approximate dynamic programming. That perspective is useful because it emphasizes the operator being approximated, not just the surface form of the algorithm.

Sources: <http://www.incompleteideas.net/book/RLbook2018.pdf>, <https://www.davidsilver.uk/teaching/>, <https://raw.githubusercontent.com/dennybritz/reinforcement-learning/master/DP/README.md>, <https://sites.ualberta.ca/~szepesva/papers/RLAlgsInMDPs.pdf>, https://ocw.mit.edu/courses/6-231-dynamic-programming-and-stochastic-control-fall-2015/dae6c8dca97f89b45bd0d7877a5433c9_MIT6_231F15_Notes.pdf

4. Monte Carlo, Temporal-Difference Learning, and Basic Control

When the environment model is unknown, the agent can learn directly from sampled experience. **Monte Carlo** methods wait until an episode ends and then use the observed return to estimate values. They are conceptually simple and unbiased given enough data, but they have high variance and fit episodic tasks most naturally.

Temporal-difference learning, or TD learning, sits between dynamic programming and Monte Carlo. Like Monte Carlo, it is model-free and learns from sampled transitions. Like dynamic programming, it bootstraps by using a current estimate inside its target. The simplest TD update moves $V(S_t)$ toward $R_{t+1} + \gamma V(S_{t+1})$, so learning can happen online after every transition instead of waiting until the end of the episode. The standard tradeoff is that TD introduces bias through bootstrapping but often reduces variance enough to be much more practical.

For control, these ideas lead to algorithms such as Monte Carlo control, **SARSA**, and **Q-learning**. SARSA is on-policy because it learns the value of the behavior it is actually following. Q-learning is off-policy because it learns toward the greedy target $R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$ whether or not the greedy action was taken. Both depend on continued exploration, usually through epsilon-greedy behavior that becomes increasingly greedy over time.

Sources: <http://www.incompleteideas.net/book/RLbook2018.pdf>,
<https://raw.githubusercontent.com/dennybritz/reinforcement-learning/master/MC/README.md>,
<https://raw.githubusercontent.com/dennybritz/reinforcement-learning/master/TD/README.md>,
<https://www.davidsilver.uk/teaching/>

5. Approximation, Generalization, and Why Tabular Methods Stop Scaling

Exact tabulation is only realistic in small discrete problems. Once the state-action space grows, the table becomes too large to store and too sparse to estimate well. More importantly, a table cannot generalize. If the agent has never visited a state before, the table has no principled way to infer a useful value for it from related states.

This is where **function approximation** enters. Instead of storing exact values for every state or state-action pair, the learner represents values with a structured function class. In classical RL this could mean linear features, aggregation, kernels, local averaging, tile coding, Fourier bases, or least-squares approximations. In deep RL it often means neural networks. The real question is not simply whether the function class is expressive, but whether it behaves well under repeated Bellman updates.

Szepesvari and Bertsekas both emphasize that approximation creates a tradeoff between approximation error and estimation error. Richer classes can fit more complicated value functions, but they can also overfit or behave poorly under Bellman operators. That is why theory-heavy RL spends so much time on projected fixed points, non-expansive approximation classes, aggregation, and error propagation. Neural networks are only one part of a broader approximation story.

Sources: <https://sites.ualberta.ca/~szepesva/papers/RLAlgsInMDPs.pdf>,
https://ocw.mit.edu/courses/6-231-dynamic-programming-and-stochastic-control-fall-2015/dae6c8dca97f89b45bd0d7877a5433c9_MIT6_231F15_Notes.pdf,
<https://www.mit.edu/people/dimitrib/RLbook.html>,
<http://www.incompleteideas.net/book/RLbook2018.pdf>

6. Deep Value-Based Reinforcement Learning

Deep RL extends the value-learning picture by replacing tables with neural networks. In **DQN**, the network $Q(s, a; \theta)$ takes a state as input and outputs one Q-value per discrete action. Conceptually, DQN is still Q-learning. The Bellman target and the control logic remain the same. What changes is that the representation now generalizes across states instead of storing an isolated value at each table entry.

The main benefit is scale. High-dimensional observations such as pixels can now be handled directly. The main cost is instability. A network update at one state-action pair changes predictions elsewhere, so the neat convergence behavior of tabular Q-learning no longer applies. Berkeley's notes, the Francois-Lavet monograph, and even simple FrozenLake-to-DQN tutorials all stress that deep Q-learning is not stable by default.

That is why DQN relies on **replay buffers** and **target networks**. Replay breaks temporal correlation by training on randomly sampled past transitions instead of only the newest sample. Target networks slow the motion of the Bellman target by keeping a delayed copy of the Q-network for target construction. In modern value-based and actor-critic deep RL, many practical tricks can be understood as attempts to stabilize the interaction between bootstrapping, off-policy data, and nonlinear function approximation.

Sources: <https://spinningup.openai.com/>, <https://rail.eecs.berkeley.edu/deeprlcourse/>, <https://www.nowpublishers.com/article/Download/MAL-071>, <https://github.com/dennybritz/reinforcement-learning>, <https://avandekleut.github.io/dqn/>

7. Policy Gradients, REINFORCE, Actor-Critic, PPO, and SAC

Value-based methods infer a policy from values. **Policy-gradient** methods optimize the policy directly. The core idea is to increase the probability of actions that lead to higher return and decrease the probability of actions that lead to lower return. The log-derivative trick makes this possible by rewriting the gradient of expected return as an expectation involving $\nabla \log \pi(a | s)$.

The simplest instance is **REINFORCE**, which uses full-trajectory returns to estimate the policy gradient. REINFORCE is conceptually clean and unbiased, but its variance is high because each action is weighted by a noisy Monte Carlo return. Practical policy-gradient methods therefore introduce reward-to-go, baselines, or learned critics to reduce variance.

This leads to **actor-critic** methods, where the actor is the policy and the critic is a learned value estimator. The critic provides a lower-variance learning signal, often through TD-style bootstrapping, so the actor does not have to rely only on full returns. Modern algorithms such as **PPO** and **SAC** build on this pattern in different ways. PPO constrains how much the policy can change in one update, usually through clipping or trust-region-inspired control of policy drift. SAC combines off-policy actor-critic learning with entropy regularization, encouraging exploration by optimizing reward plus policy entropy. These methods are central in continuous control and in modern policy optimization more broadly.

Sources: <https://spinningup.openai.com/>, <https://rail.eecs.berkeley.edu/deeprlcourse/>, <https://www.nowpublishers.com/article/Download/MAL-071>, <http://www.incompleteideas.net/book/RLbook2018.pdf>

8. Exploration as a Statistical and Control Problem

Exploration is often presented in beginner material as "take random actions sometimes," but the theory-oriented resources make a deeper point. In RL, the agent chooses the data distribution by choosing actions, so exploration is not just noise injection. It is a statistical and control problem about how to gather information while still achieving good reward.

At the heuristic level, epsilon-greedy and Boltzmann exploration remain common because they are simple and often effective. At a more principled level, methods such as UCB and optimism-based exploration treat uncertainty itself as part of the decision rule. Sutton and Barto also connect this to the older control-theoretic language of identification versus control, sometimes called dual control, where the agent must both perform well and learn about the system at the same time.

In deep RL, exploration is often absorbed into entropy bonuses, stochastic policies, intrinsic rewards, goal relabeling, or schedule design. The underlying issue is the same across all of these views: the learner must decide which parts of the environment to sample in order to improve both its model of the world and its long-run behavior.

Sources: <http://www.incompleteideas.net/book/RLbook2018.pdf>,
<https://sites.ualberta.ca/~szepesva/papers/RLAlgsInMDPs.pdf>, <https://spinningup.openai.com/>,
<https://rail.eecs.berkeley.edu/deeprlcourse/>

9. Reward Design, Approximate Dynamic Programming, Rollout, and Optimal Control

The control-oriented sources stress that RL is not only about neural training pipelines. It is also a form of stochastic optimal control. In that language, the central objects are policies as feedback laws, cost-to-go functions, Bellman equations, rollout procedures, and stability or improvement guarantees.

Approximate dynamic programming, or ADP, is the umbrella idea for preserving Bellman structure when exact DP is impossible. Approximation may appear in the cost-to-go, the policy class, the state aggregation, the minimization step, or the expectation calculation, but the goal is still to preserve the logic of dynamic programming as much as possible. Bertsekas organizes RL this way and treats many modern-looking methods as cases of approximate policy iteration or approximate value iteration.

One especially useful control idea is **rollout**. Start from a base policy or heuristic, replace the unknown tail cost with the base policy's cost-to-go, perform one-step or multistep lookahead, and choose the first action with the best immediate cost plus approximated continuation value. This can be read as one policy-improvement step from a base policy. That connects RL to heuristic search, model predictive control, and Monte Carlo tree search rather than isolating it as a separate subject.

Sources: <https://www.mit.edu/people/dimitrib/RLbook.html>, https://ocw.mit.edu/courses/6-231-dynamic-programming-and-stochastic-control-fall-2015/dae6c8dca97f89b45bd0d7877a5433c9_MIT6_231F15_Notes.pdf, https://faculty.engineering.asu.edu/bertsekas/wp-content/uploads/sites/129/2019/10/RL_2-SHORT-INTERNET-POSTED.pdf, <https://sites.ualberta.ca/~szepesva/papers/RLAlgsInMDPs.pdf>

10. What the Library Links Reveal About Practical RL Systems

The library links also teach reinforcement learning content, but indirectly, through the workflows they expose. Across RLlib, MushroomRL, Baselines, and Coach, the same algorithm families reappear: value-based methods, policy-gradient and actor-critic methods, continuous-control actor-critic methods, imitation-learning methods, and replay-augmented sparse-goal methods such as HER. That is a practical lesson in itself. Modern RL workflow selection is mostly about matching the algorithm family to the regime: discrete online control often uses DQN or PPO-like methods, continuous control often uses DDPG, TD3, SAC, or PPO-like pipelines, and offline or demonstration-heavy settings introduce BC, MARWIL, FQI, or GAIL-style methods.

The environment interfaces also reveal how RL systems are organized. Simpler stacks expose a direct `reset/step` loop, while systems-heavy stacks add vectorization, multi-agent routing, replay components, and separate collection and learning processes. MushroomRL makes the training loop explicit through `Core.learn(...)` and `Core.evaluate(...)`. Coach exposes `heatup`, `training`, and `evaluation` phases. RLlib makes the same structure distributed, separating environment runners from learners and supporting replay buffers, offline datasets, and multi-agent environments as first-class components.

Replay buffers, actor-critic pipelines, distributed rollouts, offline RL data flows, and multi-agent APIs are therefore not implementation trivia. They are concrete expressions of the deeper algorithmic ideas already discussed: off-policy reuse of experience, separation between action selection and value estimation, asynchronous sampling, and the scaling problems created by modern RL workloads.

Sources: <https://docs.ray.io/en/latest/rllib/rllib-algorithms.html>, <https://docs.ray.io/en/latest/rllib/key-concepts.html>, <https://docs.ray.io/en/latest/rllib/rllib-replay-buffers.html>, <https://docs.ray.io/en/latest/rllib/rllib-offline.html>, <https://docs.ray.io/en/latest/rllib/multi-agent-envs.html>, <https://mushroomrl.readthedocs.io/en/latest/>, <https://github.com/openai/baselines>, <https://intellabs.github.io/coach/>

11. A Clean Way to Read the Linked Material

Taken together, the linked material suggests a sensible conceptual order. Start with the agent-environment loop, return, MDPs, value functions, and Bellman equations. Then study dynamic programming, Monte Carlo, TD learning, and tabular control so that generalized policy iteration becomes intuitive. After that, move to approximation and generalization, because that is the step that makes deep RL necessary. Then study DQN, replay, target networks, policy gradients, actor-critic, PPO, and SAC as specific solutions to the instability created by combining bootstrapping, off-policy data, and nonlinear approximators. Finally, use the library documentation to see how those ideas are packaged into real training systems.

That order is more faithful to the underlying subject than starting from a library or from a benchmark paper. It keeps the Bellman and control structure visible while still making room for modern deep RL and large-scale systems.

Appendix: Exact Links Extracted from the Canvas Page

Tutorials

- Basic tutorial and worked examples: <https://github.com/dennybritz/reinforcement-learning>
- Blog series providing gentle intro: <https://medium.com/emergent-future/simple-reinforcement-learning-with-tensorflow-part-0-q-learning-with-tables-and-neural-networks-d195264329d0>
- Intro to DeepRL algorithms: <https://spinningup.openai.com/>

Libraries

- RLlib: <https://ray.readthedocs.io/en/latest/rllib.html>
- Baselines: <https://github.com/openai/baselines>
- MushroomRL: <https://mushroomrl.readthedocs.io/en/latest/>
- Coach: <https://github.com/IntelLabs/coach>

Courses

- David Silver (UCL): <http://www0.cs.ucl.ac.uk/staff/D.Silver/web/Teaching.html>
- Sergey Levine (Berkeley): <http://rail.eecs.berkeley.edu/deeprlcourse/>
- Dimitri Bertsekas (MIT): <http://web.mit.edu/dimitrib/www/RLbook.html>

Books

- Richard Sutton and Andrew Barto: <http://www.incompleteideas.net/book/RLbook2018.pdf>
- Vincent Francois-Lavet et al.: <https://www.nowpublishers.com/article/Download/MAL-071>
- Csaba Szepesvari: <https://sites.ualberta.ca/~szepesva/papers/RLAlgsInMDPs.pdf>